Methods in Deep Reinforcement Learning Used to Play Kirby's Dream Land.

Lucas Hasting

University of North Alabama
Faculty Mentor: Dr. Mark Terwilliger

MA 395
April 15, 2026

Outline

- Introduction
- Introduction to Machine Learning
- Introduction to Deep Reinforcement Learning
- Past Work
- The Project
- Future Work
- References

Kirby's Dream Land

- Kirby's Dream Land is an arcade platformer developed by HAL Laboratory and published by Nintendo for the Nintendo Game Boy.
- The game was proposed and directed by Masahiro Sakurai, a new employee at HAL Laboratory/Nintendo at the time [5].
- I choose this game since it is simple for humans to play, short, and is complex enough to study for the purposes of reinforcement learning.
- I started a similar project in AI (CS 470) under Dr. Terwilliger and wanted to learn more about Deep Learning, specifically in the context of video games.

Project Goal

- This project investigates contemporary deep reinforcement learning algorithms and their effectiveness in training an agent to autonomously play Kirby's Dream Land (specifically the boss fights).
- From this project, I wanted to learn how a machine could learn how to play video games, using Kirby's Dream Land.

Machine Learning

- **Machine Learning** - "A subfield of artificial intelligence (AI) that gives computers the ability to learn without explicitly being programmed" [1].
- **Supervised Learning** - A subfield of machine learning dedicated to learn a probability distribution $p(y \mid \mathbf{x})$.
- In other words, we wish to predict the probability of y belonging to a specific outcome using the value of $\mathbf{x}$ (our predictors/input). This comes in two categories of output.
- **Classification** - Our outcome belongs to a class.
- **Regression** - Our outcome is some real number on the number line [3].

Deep Learning - DFNN

- Deep learning is a subfield of supervised machine learning.
- Specifically, we are approximating functions using function composition.
- These relationships are often depicted using a graph which are called a network.
- These models have an input layer (the input vector) and an output layer (the most outside function), where all functions in between are referred to as hidden layers [3].

DFNN Example

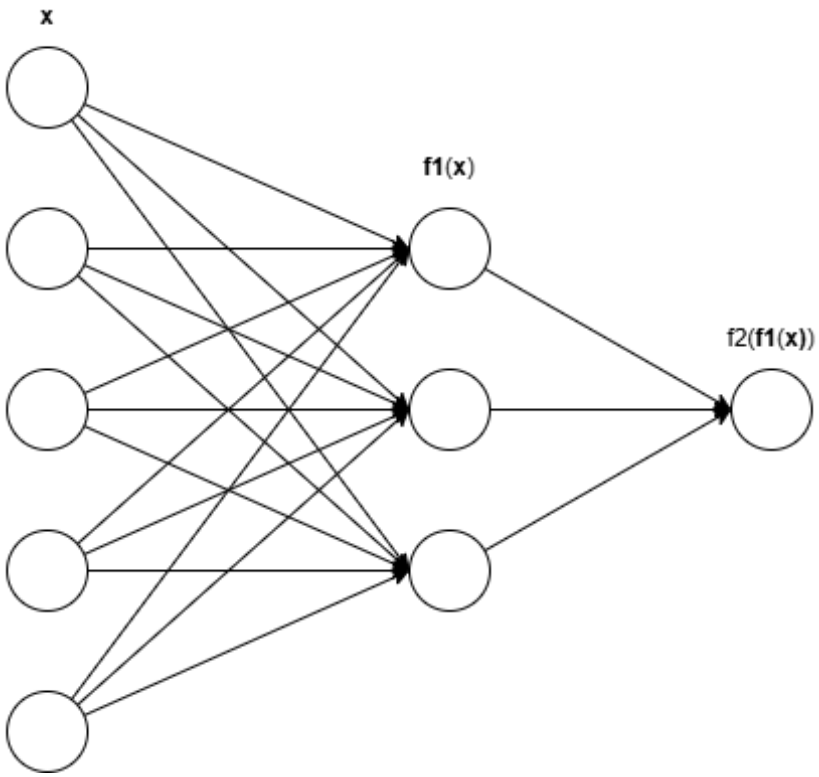

Figure 1: DFNN

DFNN Example Functions

$$\mathbf{ReLU}_i(\mathbf{x}) = \max\{0, x_i\}$$

$$\sigma(x) = \frac{1}{1 + e^{-x}}$$

$$\mathbf{f_1}(\mathbf{x}) = \mathbf{ReLU}\left(\mathbf{x}\begin{bmatrix} -1.3321 & -4.2124 & -1.9546 \\ -4.2729 & 6.1888 & -2.3244 \\ 1.3101 & -5.3542 & 2.1324 \\ 0.3361 & 1.8541 & 0.0944 \\ -1.4612 & 1.2690 & -2.7183 \end{bmatrix} + \begin{bmatrix} 3.6913 \\ -0.0730 \\ 3.9457 \end{bmatrix}^T\right)$$

$$f_2(\mathbf{x}) = \left[\left[\sigma(\mathbf{x}\begin{bmatrix} -6.0766 \\ -9.3592 \\ -6.0285 \end{bmatrix} + 5.3688)\right]\right]$$

Activation Functions

- When we create Deep Networks, we often use a linear function, and pass that through a non-linear function (to find non-linear relationships), referred to as activation functions.
- We specifically use the ReLU (Rectified Linear Units) activation function, defined below:

$$\mathbf{ReLU}_i(\mathbf{x}) = \max\{0, x_i\} \text{ [3]}.$$

The Convolution

- In our model, we also use convolution layers.
- A convolution uses a kernel (which acts as a replacement for weight), and an input matrix to perform the operation [3, 4].

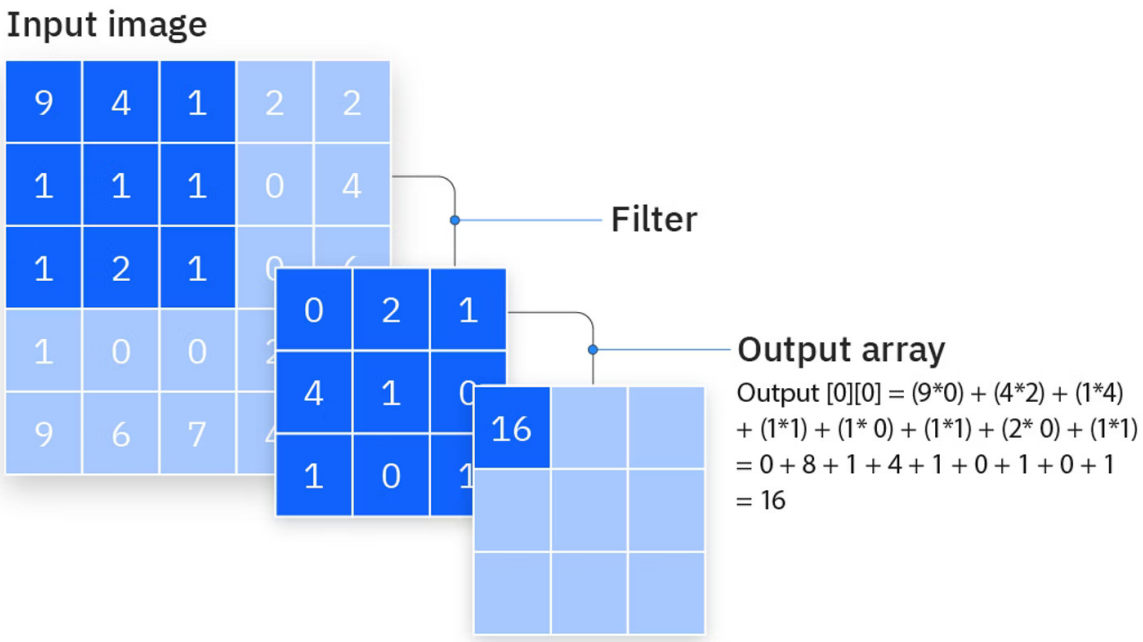

Figure 2: Convolution

The Convolution Operation

$$(I * K)_{i,j} = \sum_m \sum_n I_{m,n} K_{i-m,j-n} \text{ [3]}.$$

Optimization and Loss Functions

- In order to make out models better, we optimize using a loss function.
- A loss function is a function used to measure how bad of a prediction we made was.
- We optimize by taking the gradient (a vector pointing towards the slope of the function) of our loss function and adjusting the weights until we get as small as we can (this is called a local optimum) [3].

ADAM Optimizer

$$W_{t+1} = W_t - \alpha \frac{\frac{m_t}{1-\beta_1^t}}{\sqrt{\frac{v_t}{1-\beta_2^t}} + \epsilon}$$

$$m_t = \beta_1 m_{t-1} + (1-\beta_1)\nabla L(\theta)$$

$$v_t = \beta_2 v_{t-1} + (1-\beta_2)(\nabla L(\theta))^2 \text{ [2].}$$

Reinforcement Learning

- Reinforcement Learning is a subfield in machine learning, which makes use of an agent, that interacts with an environment and learns based on a reward function.
- A reward function is a function that describes how good the agent was at whatever task it was performing.
- This approach to machine learning is common with learning how to play games, where an agent acts as a player [10].

The Q-Learning Algorithm

- The Q in Q-Learning stands for Quality, which determines the quality of a move.
- The algorithm builds a table using previous Q values and the current state to estimate the next value of Q after a given action, we will denote this value as Q^*.
- Initial values of the table are created at random, this is referred to as exploring the environment.
- This algorithmic approach is referred to as dynamic programming.

$$Q^* = Q + \alpha R(s, a) + \gamma(\max(Q) - Q) \text{ [10].}$$

Deep Q-Network

- A DQN makes used of two neural networks, and the Q-Learning Algorithm.
- The table of Q-values is referred to as the experience-replay buffer, and is used to give us our actual reward value, in this project we only maintain the 64 most recent entries.
- The online network makes use of the bellman equation, the experience-replay buffer, and target network to update the weights of the network using this estimation [10].

Deep Q-Network

- Both networks have the same architecture, where the input is the current state and the output is every Q-value associated with every possible action.
- The target network copies the parameters of the online network every N steps, this is done to compute the result of the loss function:

$$L(\theta) = (Q^* - Q(\theta))^2 \text{ [10]}.$$

Double Deep Q-Network

- The Q-Learning Algorithm has an overestimation problem with estimating Q-values.
- The DQN inherits this overestimation problem, this problem is solved with the DDQN (Double Deep Q-Network).
- Q^* is estimated in the following way:

$$Q^* = Q + \alpha R(s, a) + \gamma(\hat{Q} - Q)$$

- $\hat{Q}$ is the highest Q value for the action, a [10].

Gym Retro

- **Legacy** - Used to refer to a system/library that is no longer maintained (no updates or bug fixes).
- Gym Retro is a *legacy* library used for interacting with several emulators.
- It was developed by OpenAI and discontinued in 2020 [9, 8].
- Since it is *legacy*, some problems arise with libraries are solutions are shown in the project repository.

Reinventing the Wheel

- Previous work on this project had me reinventing the wheel using linear and logistic regression.
- Logistic regression was used to predict a move made.
- Linear regression was used to predict how long the move should be made.

Reinventing the Wheel

- The models were updated whenever a "good move" was made, which was decided by a set of conditional statements.
- The problem encountered was that it became a mostly logical based system with very little learning actually occurring.

The Atari Project

- There was a research project conducted by DeepMind Technologies where a DQN was used to try to play several arcade Atari games.

	B. Rider	Breakout	Enduro	Pong	Q*bert	Seaquest	S. Invaders
Random	354	1.2	0	−20.4	157	110	179
Sarsa [3]	996	5.2	129	−19	614	665	271
Contingency [4]	1743	6	159	−17	960	723	268
DQN	**4092**	**168**	**470**	**20**	**1952**	**1705**	**581**
Human	7456	31	368	−3	18900	28010	3690
HNeat Best [8]	3616	52	106	19	1800	920	**1720**
HNeat Pixel [8]	1332	4	91	−16	1325	800	1145
DQN Best	**5184**	**225**	**661**	**21**	**4500**	**1740**	1075

Figure 3: Results [7]

A Similar Project

- A similar project was conducted by Pedro Alves and Andreas Høiberg Eike from the University of Agder.
- They used a new system called PyBoy and made some adjustments to an existing tutorial [10].

A Tutorial

- A tutorial was followed as a start to the project.
- The tutorial provided the following wrappers:
  - **GrayScaleObservation** - transforms screen data from RGB values to a gray scaled value (lossy compression) to reduce the size of the data.
  - **ResizeObservation** - transforms the gray scaled values into a smaller range.
  - **SkipFrame** - adds an option to skip a specific number of frames per second (determined by a parameter, default is 1).
  - **FrameStack** - stacks screen data into 4 2D arrays (also called a matrix), which will be used for our model.
  - (Extra) **ResetCompatWrapper** - Used to override the behavior of reset (used to start a new episode) to be compatible with my version of Gym Retro [11].

A Tutorial

- The tutorial also provided starter code for a DQN which was transformed into a DDQN by Alves and Eike.
- The hyper-parameters chosen for the models were re-used from their project [10].

Modifying and Adjusting

- The system Alves and Eike used interacts with the emulation environment differently from Gym Retro, so a modified reward function was used.
- A CUDA GPU was used to compute matrix operations.
- The tutorial used also had wrappers for action spaces (the total actions possible), which do not exist for the Game Boy emulator used (gambatte), so an array of 15 possible actions where used [9].

Modifying and Adjusting

- An action is a set of button(s) pressed on the Game Boy represented with 1 (pressed) and 0 (not pressed). These values had to be manually found.
- Moreover, due to the size of the input, instead of using tensors entirely throughout, tensors had to be converted to numpy arrays when caching (used for the experience-replay buffer).

Hyper-parameters [10]

Parameter	Value
exploration rate	1
exploration rate decay	0.99999975
exploration rate min	0.01
batch size	64
save every	2×10^5
γ	0.8
α	0.0002
β_1	0.9
β_2	0.999
burnin	1000
learn every	3
sync every	100

Algorithm 1 Reward Function

Require: S, S'

```
if S["boss_health"] > 0 AND S'["boss_health"] = 0 then
    Q ← 10,000
end if
if S'["boss_health"] < S["boss_health"] then
    Q ← 1,000
end if
if S["kirby_health"] < S'["kirby_health"] AND S["kirby_health"] = 1 then
    Q ← −100
end if
if S'["kirby_health"] = 0 AND S["kirby_health"] ≠ 0 then
    Q ← −1,000
end if
if S'["score"] > S["score"] then
    Q ← 100
end if
```

The Model Architecture

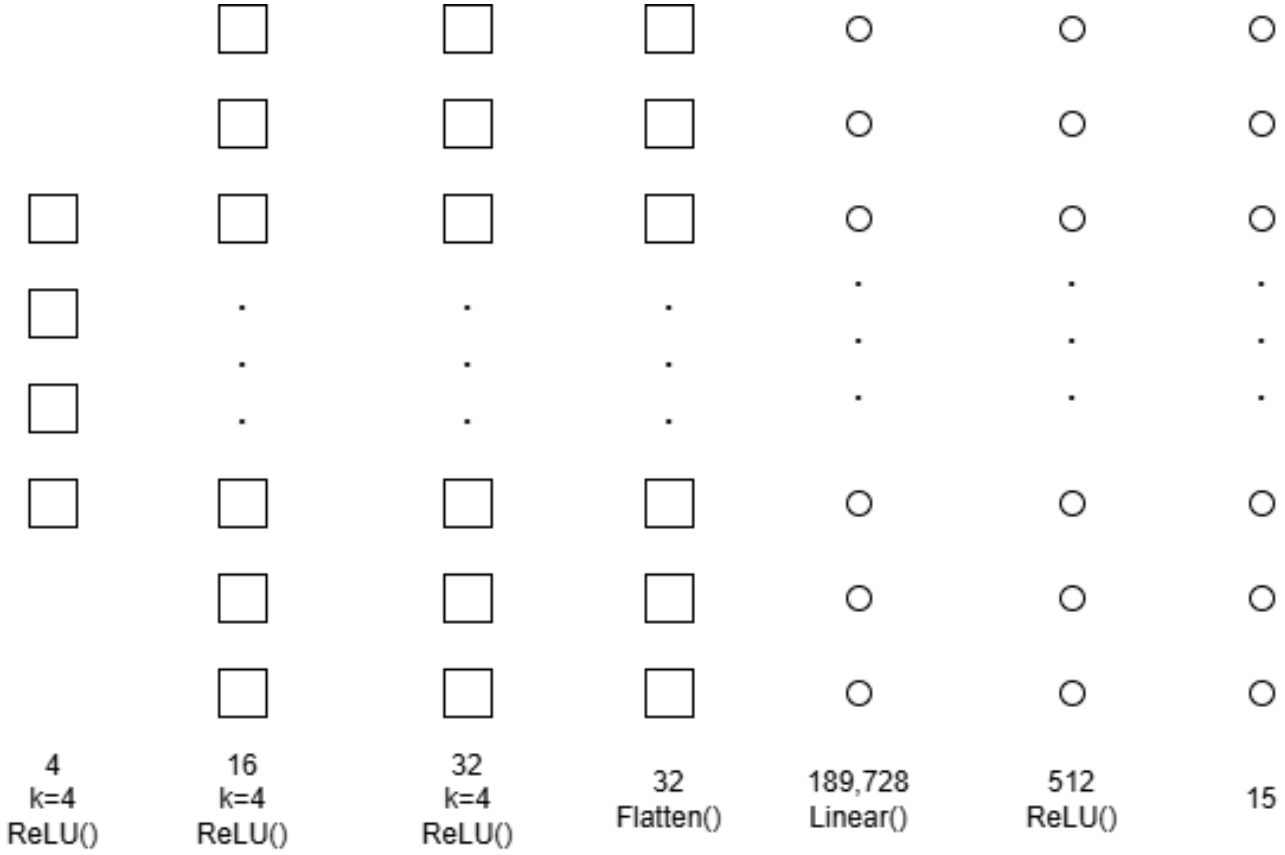

Figure 4: Model Architecture

Poppy Bros. Sr.

Figure 5: Poppy Bros. Sr.

Lololo, Lalala

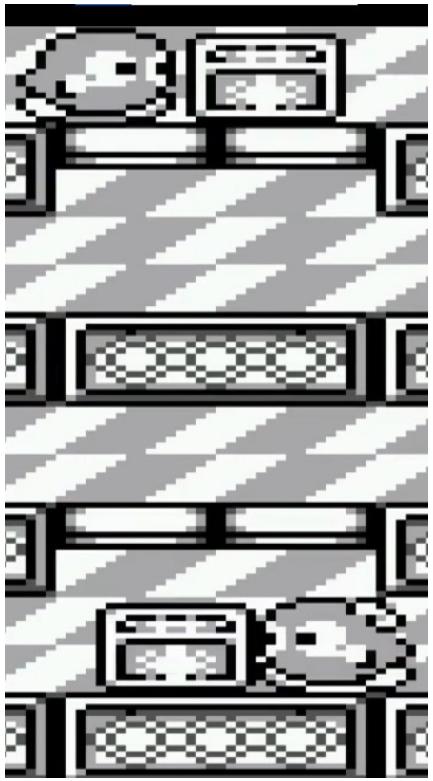

Figure 6: Lololo, Lalala

Results - Poppy Bros. Sr., Lololo, Lalala

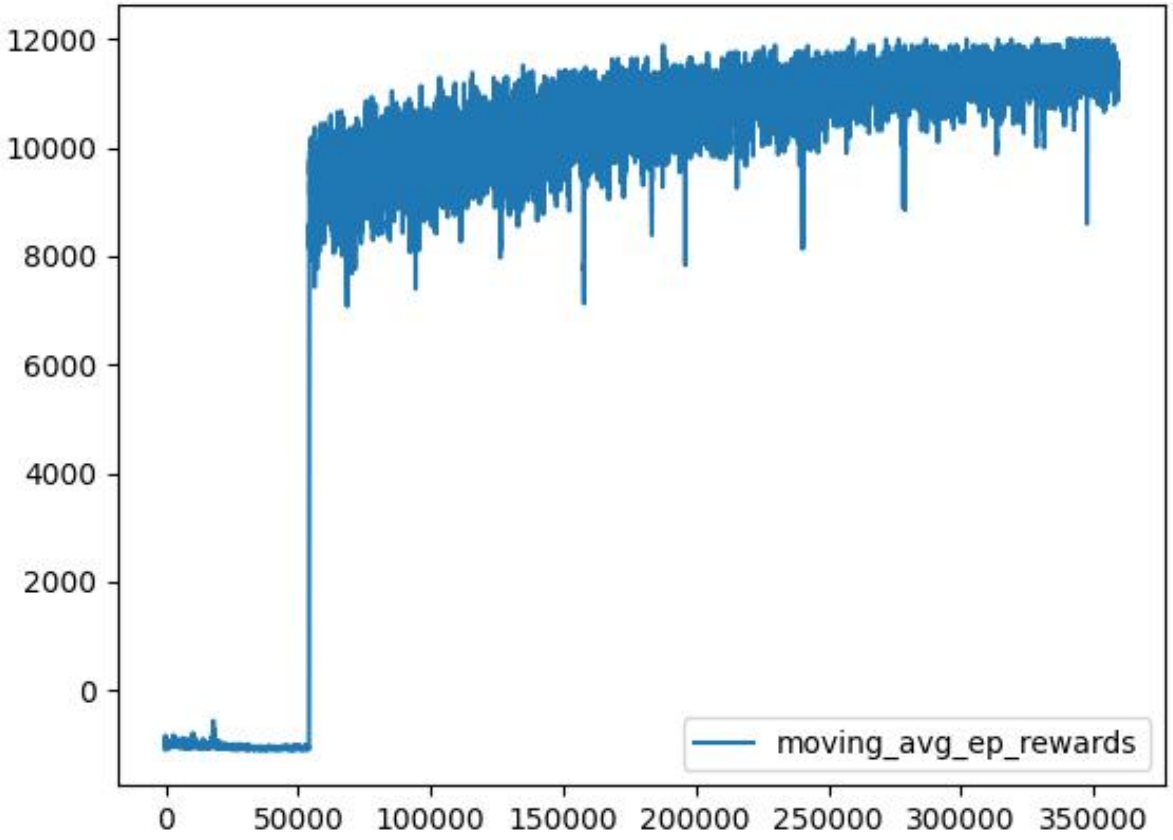

Figure 7: Reward by Episode

Whispy Woods

Figure 8: Whispy Woods

Results - Whispy Woods

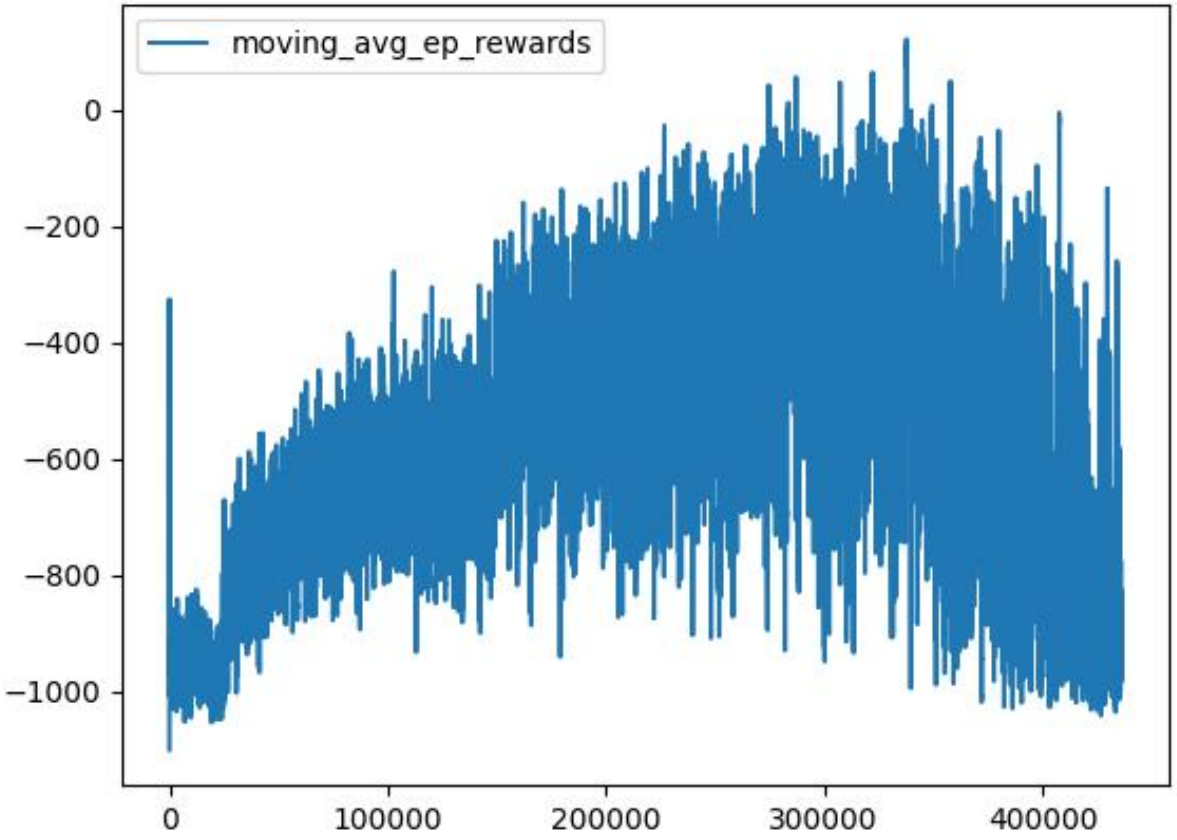

Figure 9: Reward by Episode

Kabula

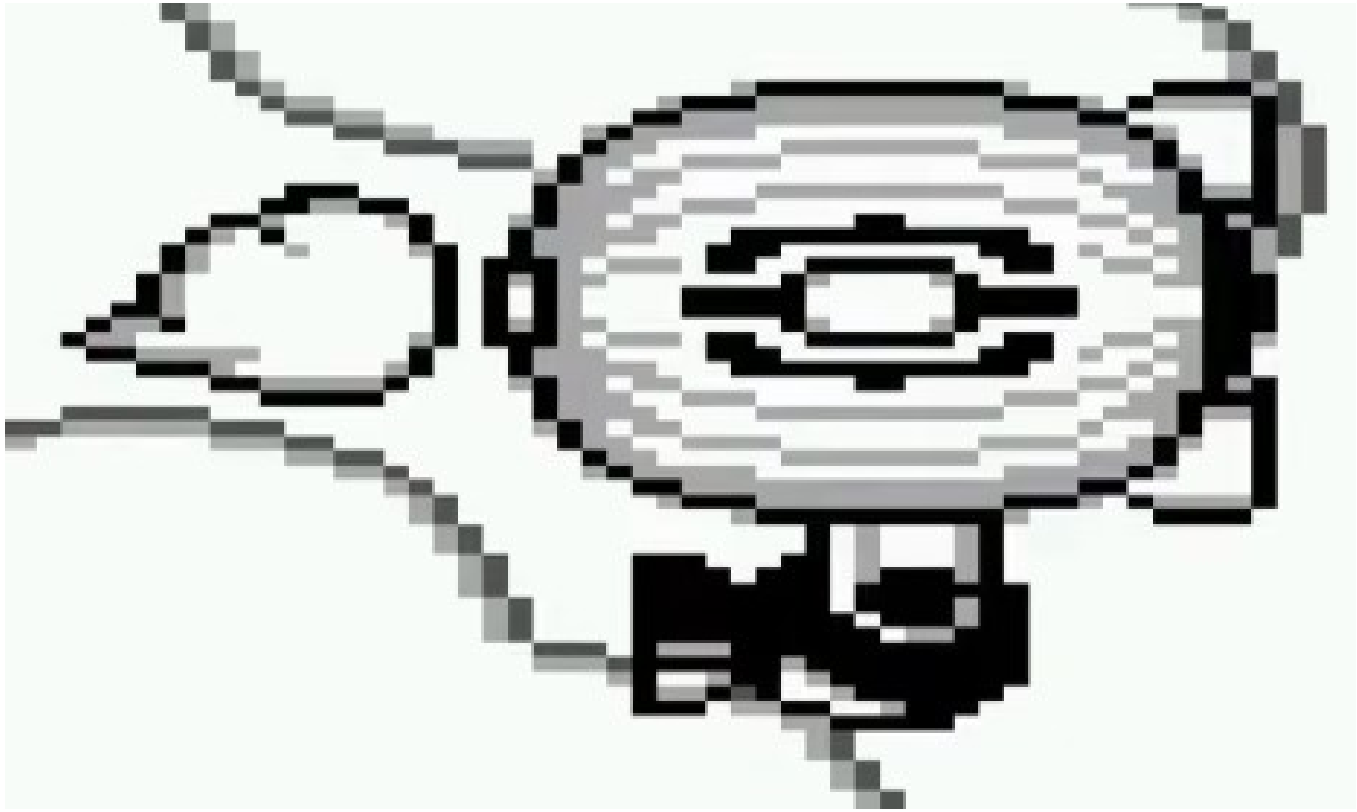

Figure 10: Kabula

Results - Kabula (Bad Attempt)

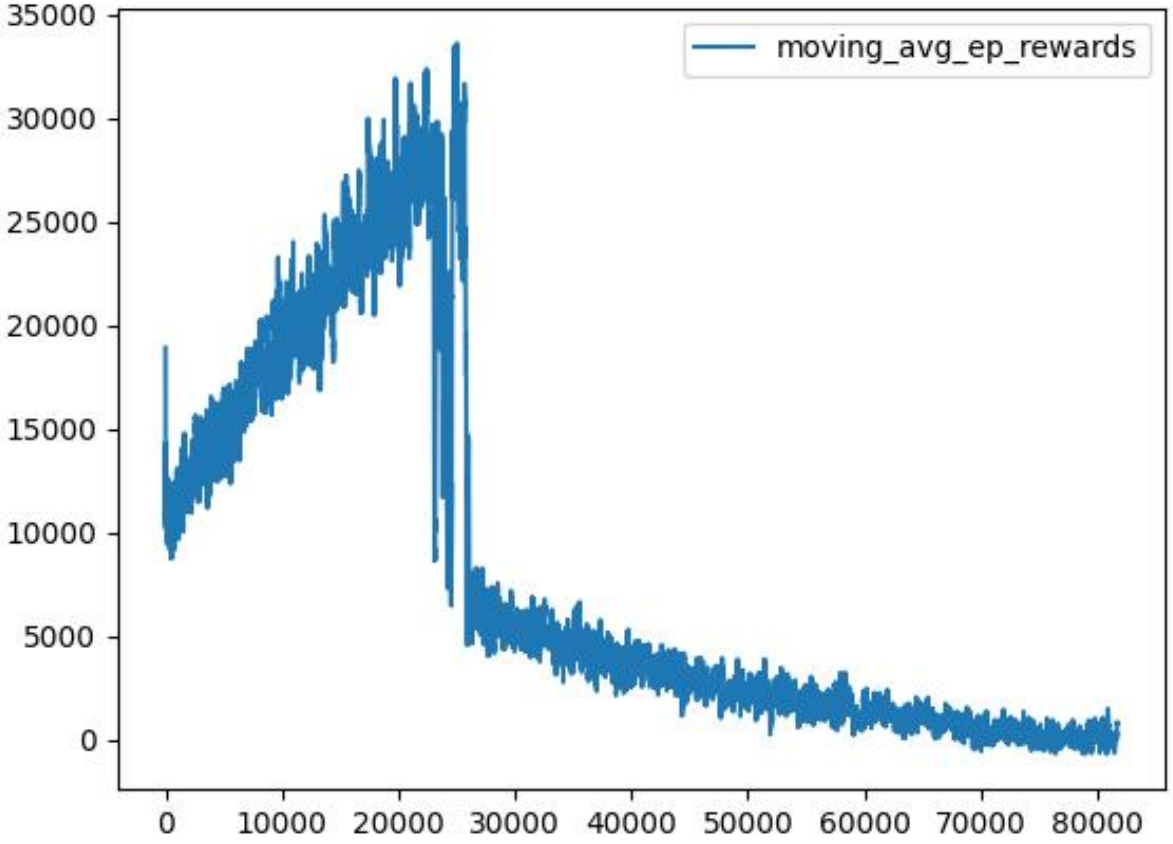

Figure 11: Reward by Episode

Results - Kabula (Good Attempt)

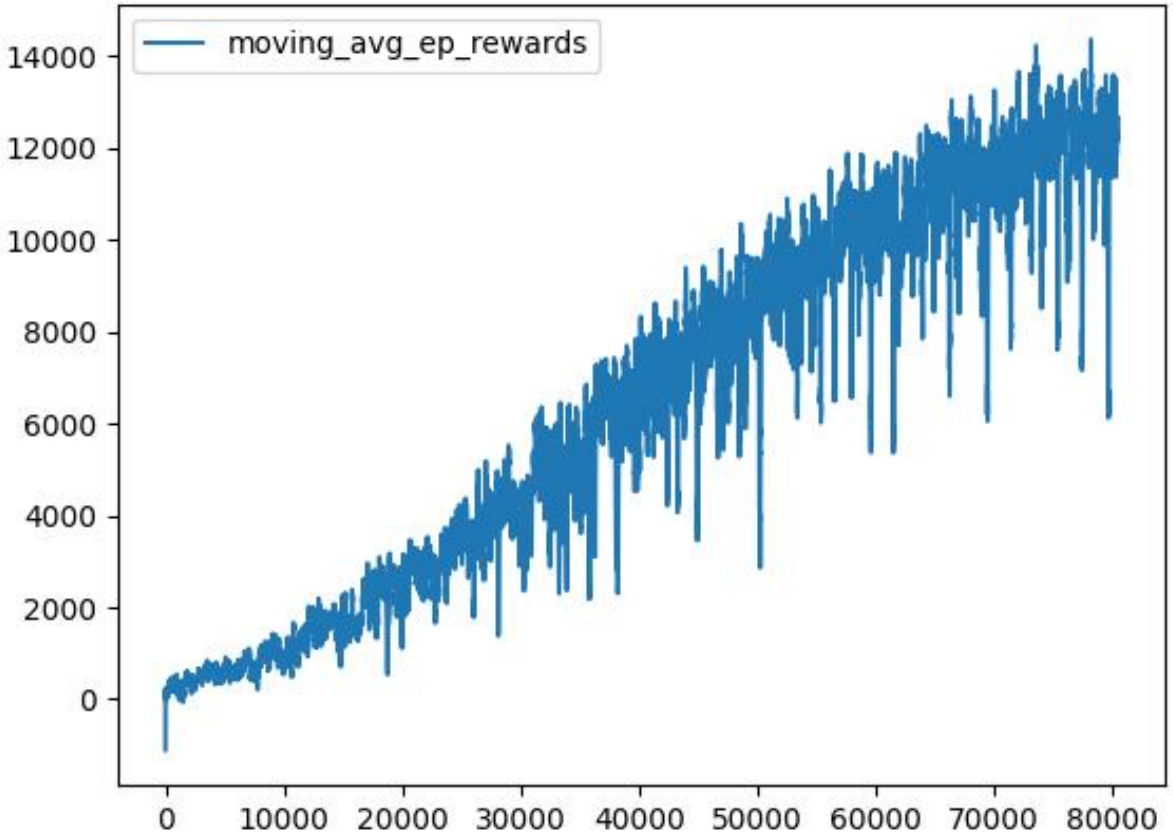

Figure 12: Reward by Episode

Kraco Jr.

Figure 13: Kraco Jr.

Results - Kraco Jr.

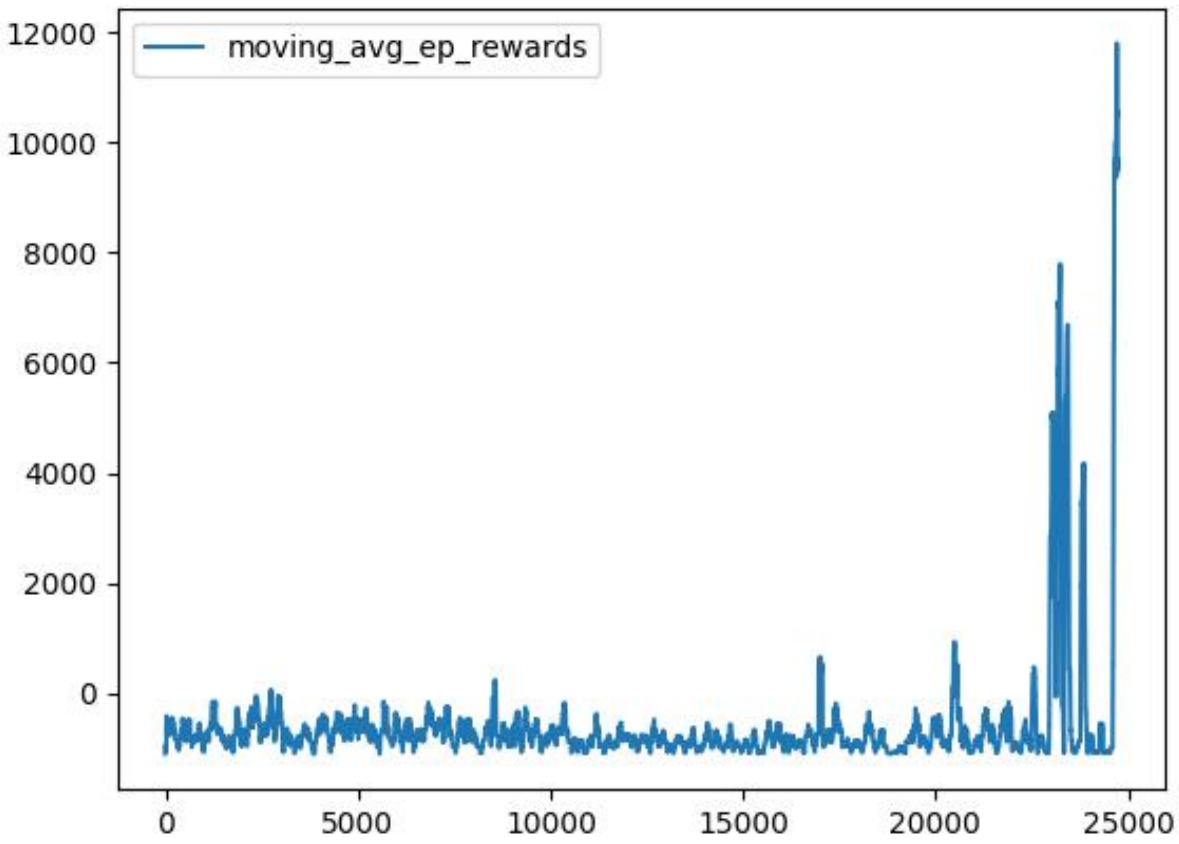

Figure 14: Reward by Episode

Kraco

Figure 15: Kraco

Results - Kraco

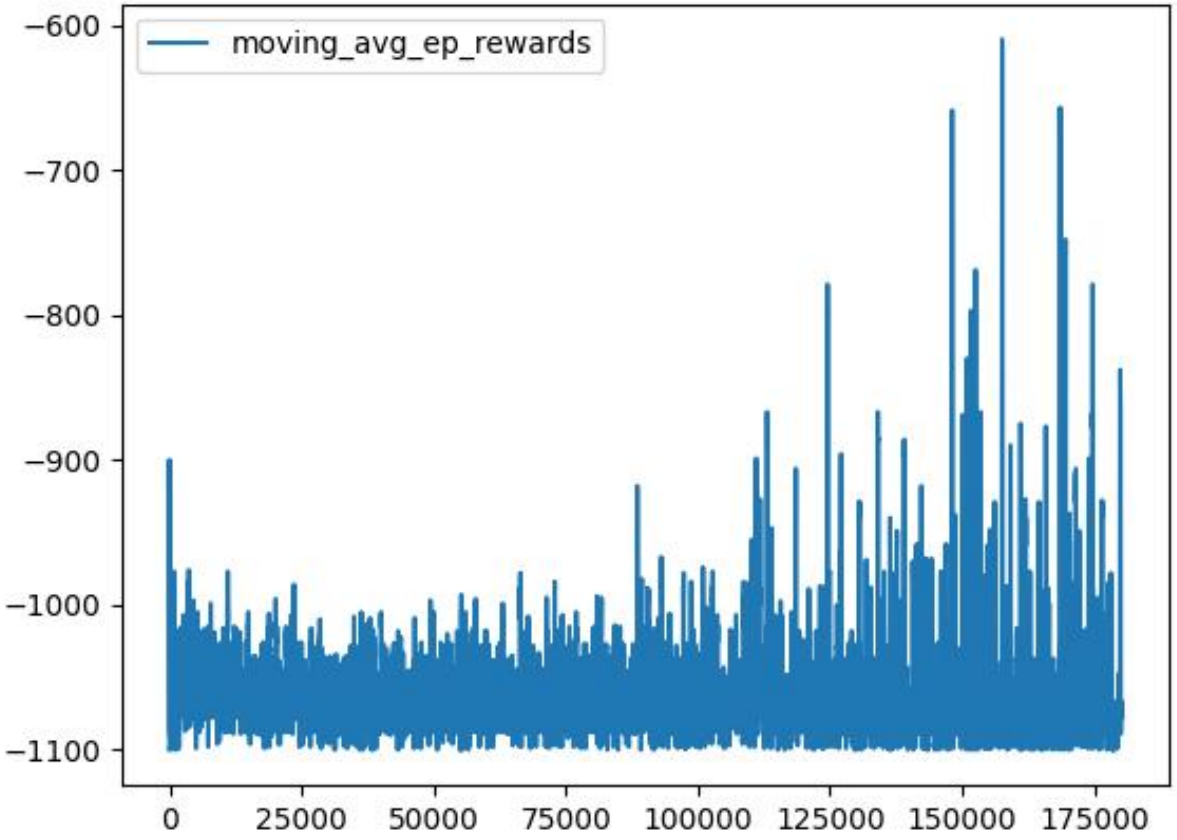

Figure 16: Reward by Episode

King DeDeDe

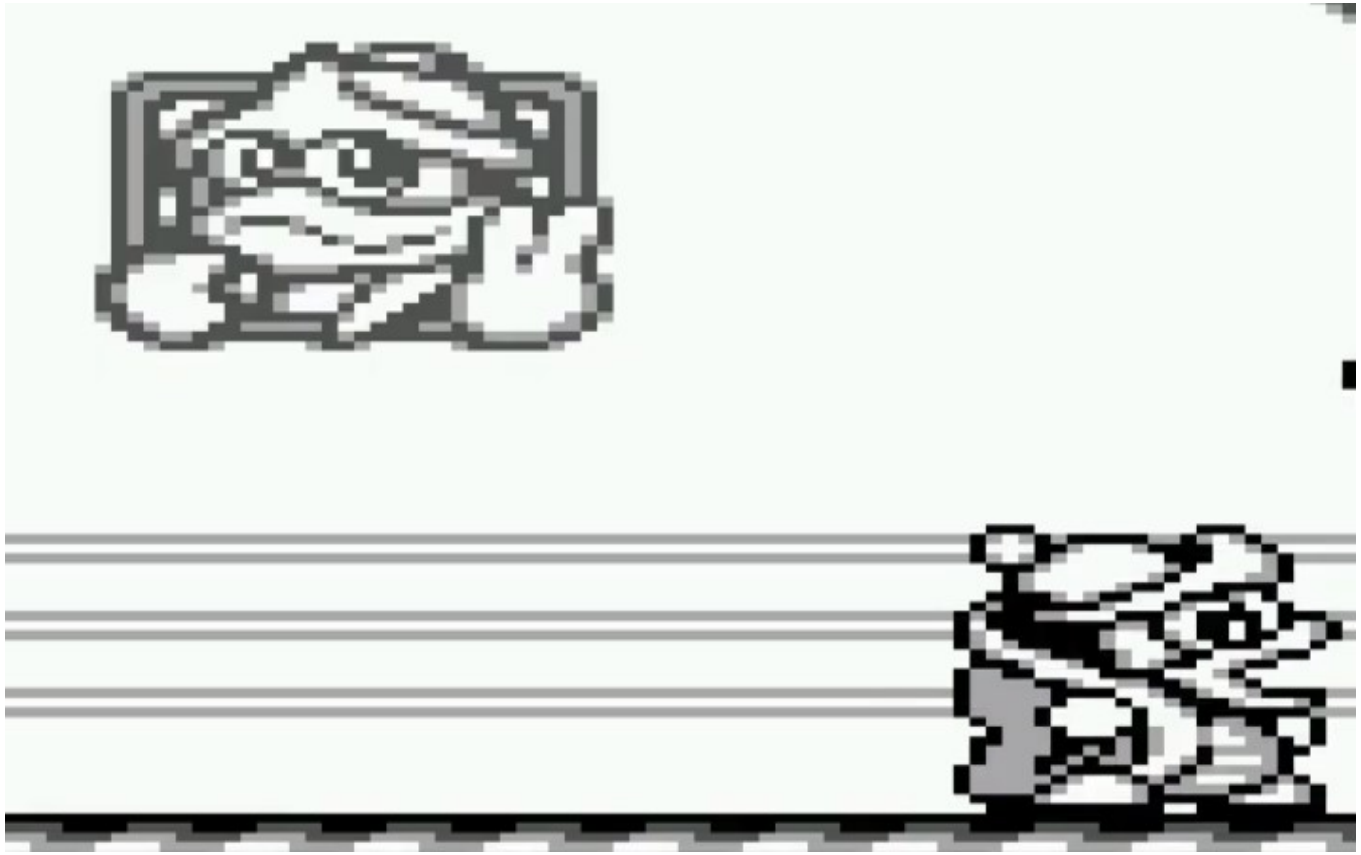

Figure 17: King DeDeDe

Results - King DeDeDe

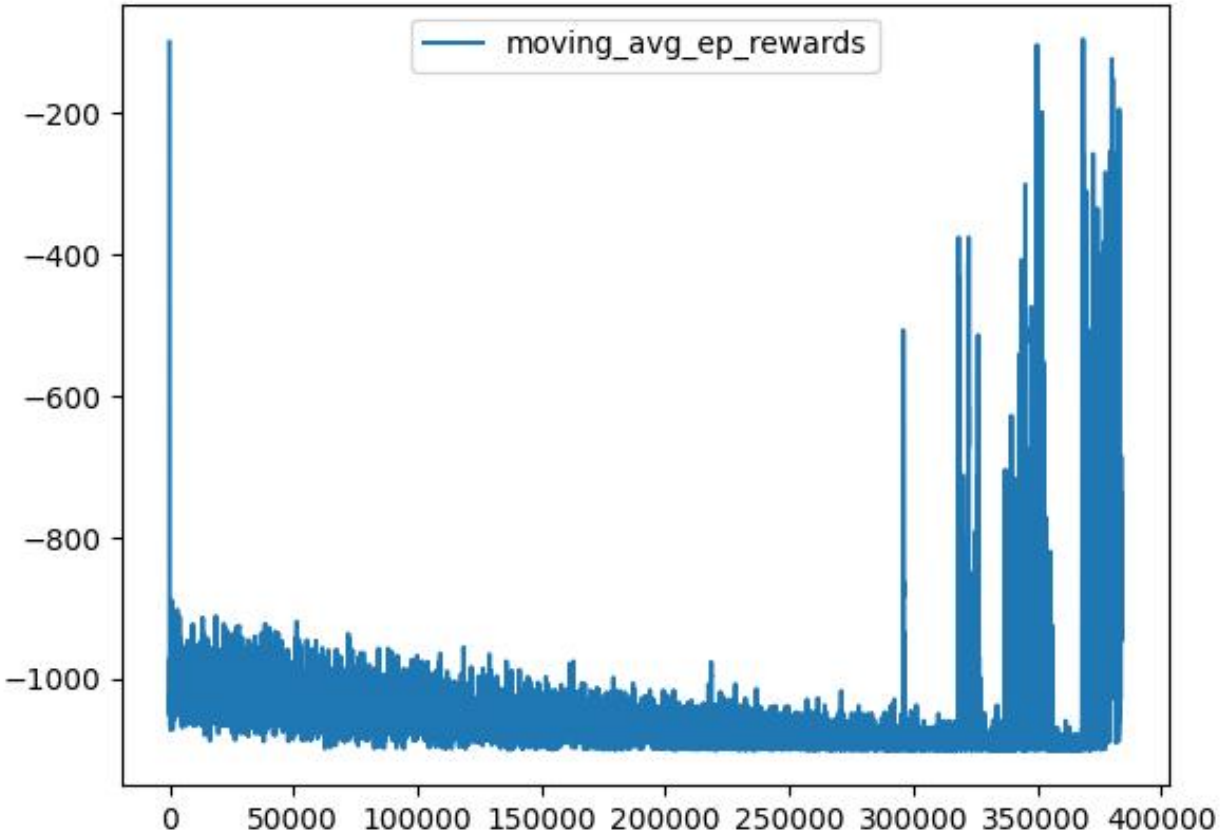

Figure 18: Reward by Episode

Demo

https://youtu.be/TSWY3JiOhro

More Agents

- Better agents can be trained for the bosses that showed poor results.
- Agents can be trained to handle different stage types (horizontal left/right, vertical up/down, enter door, and maze).
- Agents can be trained to handle each segment of the game individually (the slowest option).
- A problem for the future will be stage 2, because that stage includes a maze that could be hard for an agent to learn how to get through.

API

- An API was created in conjunction with a decision tree on data collected prior to this project in order to classify which agent would be used (for using agents trained on specific stage types).
- An API had to be used due to Python version compatibility issues with Gym Retro (which requires Python version 3.8).
- This API could be improved by exploring other models.

Conclusion

- Overall, the reward function used was mostly effective in playing Kirby's Dream Land.
- Therefore, Deep Reinforcement Learning using a DDQN was the most effective approach to learning how to play the game.
- Moreover, these techniques can be applied to learn in other areas as well, including but not limited to, autonomous robotics, data center cooling, finance, and transportation [6].

References I

Andriy Onufriyenko.
Machine learning, explained.
https://mitsloan.mit.edu/ideas-made-to-matter/machine-learning-explained.
Accessed: 2026-04-13.

Geekforgeeks.
What is adam optimizer?
https://www.geeksforgeeks.org/deep-learning/adam-optimizer/.
Accessed: 2026-04-13.

Goodfellow, I., Bengio, Y., and Courville, A.
Deep Learning.
MIT Press, 2016.
http://www.deeplearningbook.org.

References II

IBM.
What are convolutional neural networks?
https://www.ibm.com/think/topics/convolutional-neural-networks.
Accessed: 2026-04-13.

Kirby Wiki.
Kirby's dream land.
https://kirby.fandom.com/wiki/Kirby%27s_Dream_Land.
Accessed: 2026-04-13.

Li, Y.
Reinforcement learning applications.
CoRR abs/1908.06973 (2019).

References III

Mnih, V., Kavukcuoglu, K., Silver, D., Graves, A., Antonoglou, I., Wierstra, D., and Riedmiller, M. A.
Playing atari with deep reinforcement learning.
CoRR abs/1312.5602 (2013).

OpenAI.
Github for gym retro.
https://github.com/openai/retro.
Accessed: 2026-04-13.

OpenAI.
Gym retro.
https://openai.com/index/gym-retro/.
Accessed: 2026-04-13.

References IV

Pedro Alves†, Andreas Høiberg Eike†.
Reinforcement learning with pyboy.
https://raw.githubusercontent.com/lixado/PyBoy-RL/main/README/report.pdf.
Accessed: 2026-04-13.

Yuansong Feng, Suraj Subramanian, Howard Wang, Steven Guo.
Train a mario-playing rl agent.
https://docs.pytorch.org/tutorials/intermediate/mario_rl_tutorial.html.
Accessed: 2026-04-13.

The Project

Figure 19: Link